E-commerce Mandala — Documento di design

Nov 3, 2026 · @Pierre

Visione e obiettivi

Un negozio online su misura per vendere quadri mandala incisi e dipinti a mano su legno, ognuno un pezzo unico, a clienti in Italia e nell'Unione Europea. Il sito deve far percepire l'artigianalità (foto, dettagli, storia del processo) e rendere l'acquisto semplice e sicuro.

Decisioni già prese:

Tema	Scelta
Catalogo	Solo pezzi unici: ogni quadro esiste una volta, una volta venduto resta visibile come "venduto"
Mercato	Italia + UE, sito in italiano e inglese
Approccio	Full custom, codice nostro dall'inizio alla fine
Funzioni v1	Account clienti, pannello admin, blog, newsletter e coupon

Obiettivi misurabili per il lancio (da confermare):

- Tempo di caricamento della pagina prodotto sotto 2 secondi su mobile
- Pubblicare un nuovo quadro dal pannello admin in meno di 5 minuti
- Zero vendite doppie dello stesso pezzo

Funzionalità v1

La v1 copre tutto il percorso dall'ispirazione all'ordine spedito; restano fuori le funzioni che servono solo con volumi più alti.

Vetrina (cliente)

- Home con quadri in evidenza e ultimi arrivi
- Catalogo con filtri: dimensione, forma (tondo, quadrato), palette colori, prezzo, disponibile/venduto
- Pagina prodotto: galleria foto ad alta risoluzione con zoom, dimensioni in cm, essenza del legno, tecniche usate, tempo di lavorazione, foto "in ambiente" per capire la scala

- Pagina "venduto" che resta online (SEO e prova sociale) con invito a iscriversi agli avvisi per opere simili
- Ricerca testuale
- Sito bilingue IT/EN

Acquisto

- Carrello con riserva temporanea del pezzo (vedi regole sotto)
- Checkout ospite o con account
- Pagamento con carta, PayPal, Apple Pay/Google Pay tramite Stripe
- Coupon (percentuale, importo fisso, spedizione gratuita) con scadenza e limiti d'uso
- Email transazionali: conferma ordine, spedizione con tracking, consegna

Account cliente

- Registrazione e login (email + password, link magico opzionale)
- Storico ordini e stato spedizione
- Wishlist
- Indirizzi salvati

Contenuti

- Blog: storia dell'artigiano, processo di incisione e pittura, significato dei mandala
- Pagine statiche: chi siamo, FAQ, spedizioni e resi, cura del legno
- Iscrizione newsletter (double opt-in)

Pannello admin

- Gestione quadri: bozza → pubblicato → riservato → venduto → archiviato
- Upload foto multiple con ritaglio e ordinamento
- Ordini: lista, dettaglio, cambio stato, inserimento tracking, rimborso
- Coupon, articoli del blog, iscritti newsletter
- Dashboard semplice: vendite del mese, ordini da spedire

Fuori dalla v1

- Commissioni personalizzate e preventivi
- Prodotti con varianti o replicabili
- Recensioni dei clienti
- App mobile
- Spedizioni extra-UE

Regole dei pezzi unici

Un quadro può essere pagato da una sola persona: la disponibilità si blocca al checkout, non all'aggiunta al carrello, e la conferma definitiva arriva solo dal webhook di pagamento.

1. Aggiungere al carrello non blocca il pezzo. Se nel frattempo viene venduto, il carrello lo segnala e lo rimuove.
2. All'avvio del checkout il pezzo passa a "riservato" per 15 minuti, con una UPDATE condizionale sul DB (`WHERE status = 'PUBLISHED'`): se due clienti partono insieme, uno solo vince.
3. Il pagamento confermato (webhook Stripe `checkout.session.completed`) porta il pezzo a "venduto" e crea l'ordine.
4. Pagamento fallito, abbandonato o riserva scaduta: un job riporta il pezzo a "pubblicato".
5. Rimborso dopo un reso: l'admin decide se ripubblicare il quadro o archiviarlo.
6. La pagina di un pezzo venduto resta online con badge "venduto" e suggerimenti di opere simili.

Stati del quadro: `DRAFT` → `PUBLISHED` → `RESERVED` → `SOLD` → `ARCHIVED`, con `RESERVED` → `PUBLISHED` alla scadenza della riserva.

Stack tecnologico

Proposta: un'unica applicazione TypeScript full-stack su Next.js con PostgreSQL, Stripe per i pagamenti e Payload CMS integrato per pannello admin e blog. Un solo linguaggio dal database al browser, un solo deploy, e l'admin non va scritto da zero.

Livello	Scelta	Perché
Linguaggio	TypeScript	Tipi condivisi tra frontend, backend e DB
Framework web	Next.js (App Router)	Rendering lato server per SEO e velocità, API e pagine nello stesso progetto
UI	Tailwind CSS + shadcn/ui	Componenti accessibili che personalizziamo con l'estetica del brand
Admin e blog	Payload CMS	Gira dentro Next.js, genera pannello admin, upload e editor articoli dalle nostre collezioni

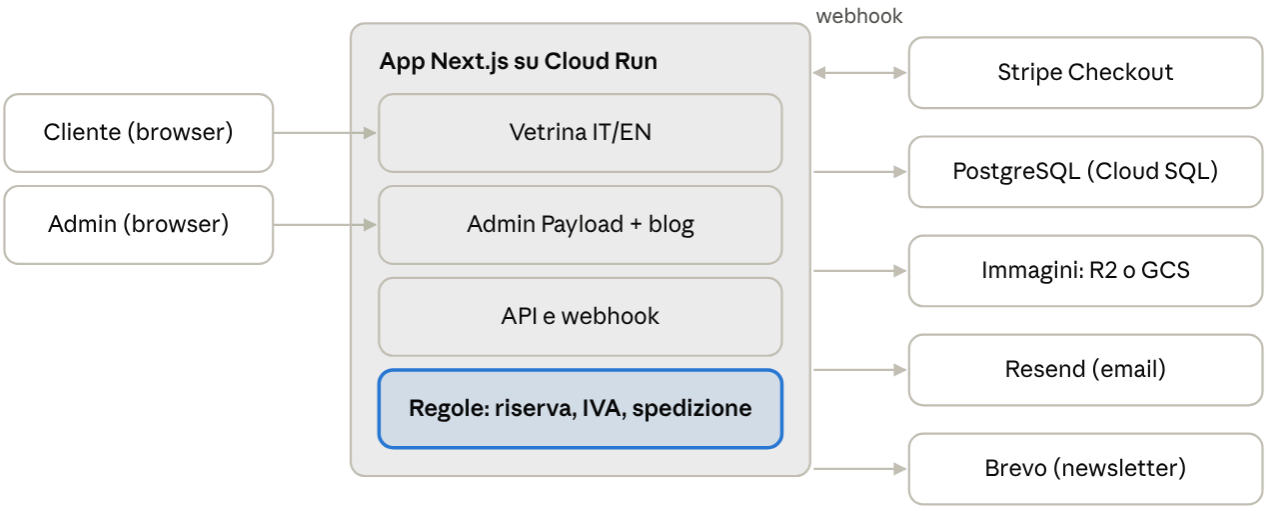
Livello	Scelta	Perché
Database	PostgreSQL	Transazioni affidabili per riserve e ordini
ORM	Drizzle (usato anche da Payload)	Query tipizzate, migrazioni versionate nel repo
Pagamenti	Stripe Checkout + webhook	Carte, PayPal, Apple/Google Pay, SCA e ricevute già gestiti
Autenticazione clienti	Better Auth	Email/password e link magico, sessioni nel nostro DB
Immagini	Object storage (Cloudflare R2 o GCS) + ottimizzazione Next/Image	Foto ad alta risoluzione servite ridimensionate
Email	Resend + React Email	Template delle email scritti come componenti
Newsletter	Brevo	Double opt-in e campagne, dati in UE
Ricerca	Full-text di PostgreSQL	Sufficiente per un catalogo di qualche centinaio di pezzi
i18n	next-intl	Rotte /it e /en
Test	Vitest + Playwright	Unit e test end-to-end del checkout
Hosting	Google Cloud Run + Cloud SQL (regione europe-west)	Dati in UE, stack che conosci già

Alternativa valutata: frontend Next.js + backend Kotlin/Spring separato. Più familiare, ma raddoppia deploy, modelli e autenticazione, e il pannello admin andrebbe scritto a mano.

Architettura

Un monolite modulare: vetrina, admin e API vivono nella stessa app Next.js e condividono le regole di dominio; Stripe è l'unico servizio che richiama l'app, tramite webhook.

Una sola app Next.js parla con database, Stripe e servizi esterni



architettura · un'app, cinque servizi

Flusso di acquisto:

1. Il cliente avvia il checkout: l'app riserva il quadro nel DB e crea una sessione Stripe Checkout.
2. Il cliente paga sulla pagina Stripe e torna alla pagina di conferma.
3. Stripe chiama il webhook: l'app verifica la firma, segna il quadro come venduto, crea l'ordine e invia l'email di conferma.
4. L'ordine compare nel pannello admin come "da spedire".

Modello dati iniziale

Il quadro è l'entità centrale e non ha varianti né quantità: lo stato sostituisce il magazzino. Gli ordini copiano prezzo e titolo del quadro al momento dell'acquisto, così restano corretti anche se il quadro viene modificato.

Entità	Campi principali	Note
Artwork	id, slug, titolo e descrizione (IT/EN), prezzo in centesimi, larghezza/altezza/spessore cm, peso g, forma, essenza legno, tecniche, palette, stato, reserved_until, published_at	Stato: DRAFT, PUBLISHED, RESERVED, SOLD, ARCHIVED

Entità	Campi principali	Note
ArtworkImage	artwork_id, url, alt (IT/EN), ordine, tipo (principale, dettaglio, ambiente)	Prima immagine = copertina
Collection	id, slug, nome (IT/EN)	Raggruppamenti tematici, molti-a-molti con Artwork
Customer	id, email, nome, lingua, consenso marketing, creato il	Gestito da Better Auth
Address	customer_id, nome, via, CAP, città, provincia, paese ISO, telefono	Spedizione e fatturazione
Wishlist	customer_id, artwork_id	
Order	id, numero leggibile, customer_id o email ospite, stato, subtotale, sconto, spedizione, IVA, totale, valuta, indirizzi copiati, stripe_session_id, tracking, timestamp	Stato: PENDING, PAID, SHIPPED, DELIVERED, REFUNDED, CANCELLED
OrderItem	order_id, artwork_id, titolo e prezzo copiati	Quasi sempre un quadro per riga
Coupon	codice, tipo, valore, minimo ordine, valido da/a, usi massimi, usi per cliente	
ShippingRate	zona (IT, UE fascia 1/2), fascia di dimensione o peso, prezzo	
Post	slug, titolo, contenuto rich text (IT/EN), copertina, pubblicato il	Collezione Payload
NewsletterSubscriber	email, lingua, stato opt-in	Sincronizzato con Brevo

I prezzi sono interi in centesimi, mai decimali; tutte le date in UTC.

Spedizioni, IVA e aspetti legali

Il codice deve supportare IVA per paese di destinazione e tariffe di spedizione per zona fin dall'inizio; le aliquote e il regime fiscale vanno però confermati con un commercialista prima del lancio.

Spedizioni

- Zone: Italia, UE fascia 1, UE fascia 2; tariffe calcolate da dimensione e peso del quadro
- Spedizione assicurata e tracciata, imballo protettivo per legno dipinto
- Integrazione con un aggregatore (Sendcloud o Packlink) in v2; in v1 tracking inserito a mano dall'admin

IVA (punti da verificare)

- Vendite a privati UE: sopra la soglia complessiva di 10.000 € annui si applica l'IVA del paese del cliente tramite regime OSS; il modello dati salva aliquota e paese per ogni ordine
- Possibile aliquota agevolata per opere d'arte vendute dall'autore: da verificare se si applica a questi quadri
- Prezzi mostrati IVA inclusa, ricalcolati al checkout in base al paese di spedizione

Obblighi per la vendita online

- Termini e condizioni di vendita, informativa sul diritto di recesso di 14 giorni
- Privacy policy GDPR e cookie banner con consenso granulare (analytics solo dopo consenso)
- Dati aziendali nel footer (ragione sociale, P.IVA, sede)
- Double opt-in per la newsletter, separato dall'accettazione dei termini
- Accessibilità: puntiamo a WCAG 2.2 AA anche se non obbligatoria per microimprese

Non sono un consulente fiscale o legale: questi punti sono la checklist da portare al commercialista.

Repository, convenzioni e ambienti

Un solo repository Git su GitHub con un'unica app Next.js; si passa a un monorepo solo se arriva un secondo servizio.

```
mandala-shop/
├─ src/
│  ├─ app/
│  │  ├─ [locale]/(shop)/     # vetrina: home, catalogo, prodotto, carrello,
account
│  │  ├─ (payload)/admin/     # pannello admin Payload
│  │  └─ api/webhooks/stripe/ # webhook pagamenti
│  ├─ collections/            # schema Payload: Artwork, Order, Post, Coupon…
│  ├─ domain/                 # regole di business: riserva, prezzi, IVA,
spedizione
│  ├─ components/
│  ├─ emails/                 # template React Email
│  └─ i18n/
├─ tests/ (unit, e2e)
├─ docker-compose.yml         # Postgres e Mailpit in locale
└─ .github/workflows/
```

Convenzioni

- Branch `main` sempre rilasciabile, feature branch e pull request
- Commit in formato Conventional Commits (`feat:`, `fix:`…)
- ESLint + Prettier, TypeScript in modalità strict
- Le regole di business stanno in `domain/` con test unitari, senza dipendere da Next.js
- Segreti solo in variabili d'ambiente, mai nel repo

Ambienti

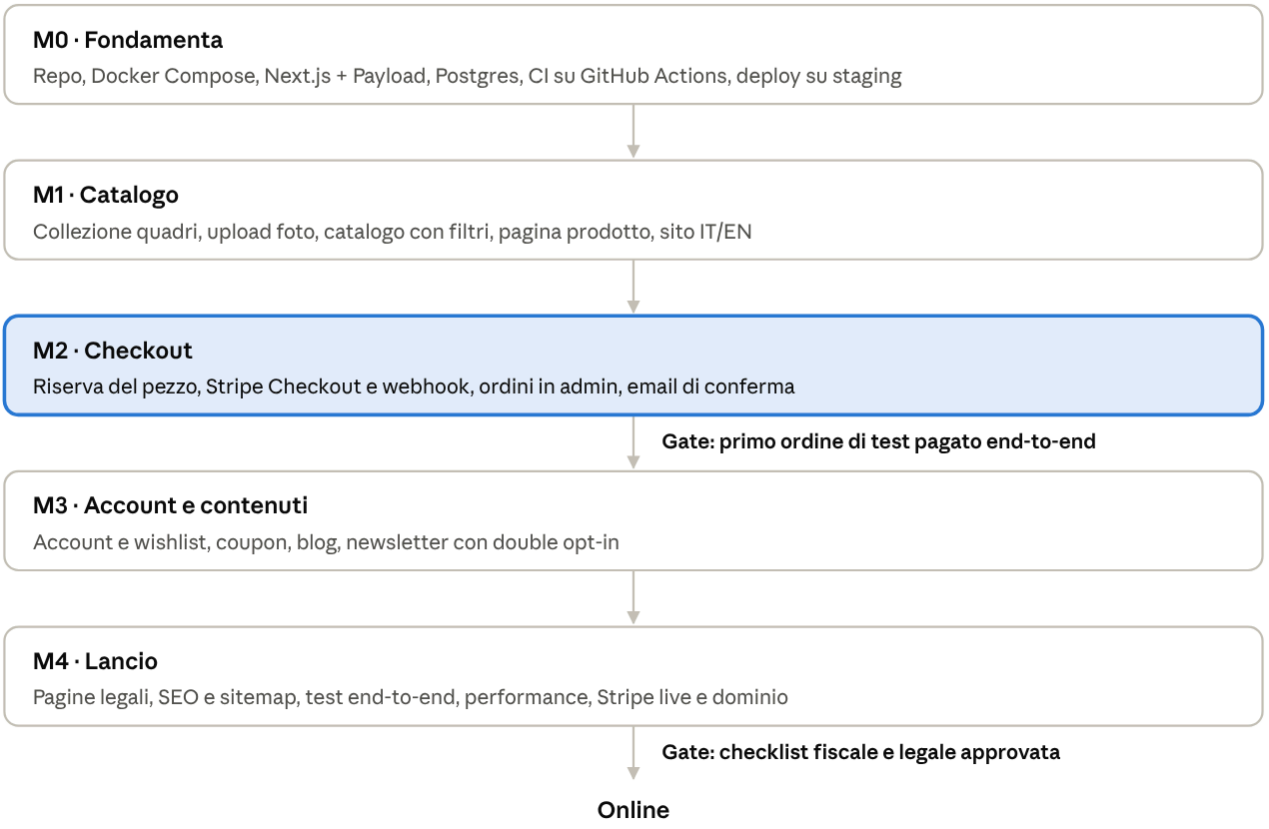
Ambiente	Dove	Pagamenti
Locale	Docker Compose sul tuo PC	Stripe test + Stripe CLI per i webhook
Staging	Cloud Run, deploy automatico da `main`	Stripe test
Produzione	Cloud Run, deploy da tag di release	Stripe live

CI (GitHub Actions): lint, type-check, test unitari, build e test end-to-end del checkout su ogni pull request.

Roadmap

Costruiamo prima il percorso che genera un ordine pagato (M0–M2), poi tutto il resto; il checkout è la fase critica.

Cinque fasi: si lancia dopo un ordine di test pagato e la checklist legale



roadmap · 5 fasi, 2 gate

Date da fissare insieme in base al tempo che possiamo dedicarci ogni settimana.

Decisioni aperte

- [ ] Confermare lo stack (Next.js + Payload) o preferire un backend Kotlin separato
- [ ] Nome del brand e dominio
- [ ] Hosting: Google Cloud (Cloud Run + Cloud SQL) o Vercel + Postgres gestito
- [ ] Storage immagini: Cloudflare R2 o Google Cloud Storage
- [ ] Regime fiscale e aliquote IVA, da chiarire con il commercialista
- [ ] Durata della riserva al checkout (proposta: 15 minuti)
- [ ] Politica resi: accettati per tutti i quadri o esclusioni
- [ ] Lingue: solo IT/EN o anche FR/DE al lancio
- [ ] Identità visiva: palette, tipografia, stile fotografico
- [ ] Quanti quadri pronti per il lancio e chi scatta le foto